

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE
AREQUIPA
FACULTAD DE INGENIERÍA, PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE CIENCIA DE LA
COMPUTACIÓN



CURSO: COMPUTACIÓN PARALELA Y DISTRIBUIDA
TEMA: LABORATORIO 2
DOCENTE: MAMANI ALIAGA, ALVARO HENRY
ALUMNO: QUINA DELGADO, MARCELO ADRIAN

FECHA: 15/09/2026

Arequipa - Perú
2026

1. Análisis de bucles anidados

1.1. Código fuente

Se presentan las dos estructuras de bucles FOR anidados propuestas por Pacheco (Capítulo 2, p. 22), que recorren las matrices/vectores A, x e y en distinto orden.

Listing 1: Versión 1 (recorrido por filas): primer par de bucles, i externo y j interno

```
1 double A[MAX][MAX], x[MAX], y[MAX];
2 /* Inicializar A y x, asignar y = 0 */
3
4 /* Primer par de bucles */
5 for (i = 0; i < MAX; i++)
6     for (j = 0; j < MAX; j++)
7         y[i] += A[i][j] * x[j];
```

Listing 2: Versión 2 (recorrido por columnas): segundo par de bucles, j externo y i interno

```
1 double A[MAX][MAX], x[MAX], y[MAX];
2 /* Inicializar A y x, asignar y = 0 */
3
4 /* Segundo par de bucles */
5 for (j = 0; j < MAX; j++)
6     for (i = 0; i < MAX; i++)
7         y[i] += A[i][j] * x[j];
```

1.2. Número de iteraciones

En ambas versiones el bucle externo se ejecuta MAX veces y, por cada iteración externa, el bucle interno se ejecuta también MAX veces. Por lo tanto, el número total de iteraciones (y de operaciones multiplicación-suma) es:

$$N_{\text{iter}} = \sum_{i=0}^{MAX-1} \sum_{j=0}^{MAX-1} 1 = MAX \times MAX = MAX^2 \quad (1)$$

Este valor es idéntico para ambas versiones: el orden de los bucles no cambia la cantidad de operaciones realizadas, solo el *patrón* en que se accede a la memoria.

1.3. Complejidad computacional

Dado que el número de iteraciones crece como MAX^2 , ambas versiones tienen complejidad temporal:

$$T(n) = O(n^2) \quad (2)$$

donde $n = MAX$. Es decir, en términos de *complejidad asintótica* ambas implementaciones son equivalentes. La diferencia de desempeño que se observará experimentalmente no proviene del número de operaciones, sino del comportamiento de la caché:

- En la Versión 1, $A[i][j]$ se accede recorriendo cada fila de forma contigua (tal como A se almacena en memoria en C, por filas), lo que favorece la localidad espacial y aprovecha mejor las líneas de caché.
- En la Versión 2, $A[i][j]$ se accede saltando de fila en fila (con paso MAX elementos), rompiendo la localidad espacial y provocando más fallos de caché (*cache misses*).

1.4. Comparación para diferentes tamaños de entrada

Se ejecutaron ambas versiones para matrices cuadradas de tamaño $n \times n$, con n igual a 256, 512, 1024, 2048, 4096 y 8192, promediando 3 repeticiones por tamaño. Los tiempos obtenidos (en segundos) se muestran en la Tabla 1.

Cuadro 1: Tiempo de ejecución (s) según tamaño de MAX			
n	Versión 1 (por filas)	Versión 2 (por columnas)	Razón v_2/v_1
256	0.00134	0.00114	0.85
512	0.00415	0.00519	1.25
1024	0.01753	0.02602	1.48
2048	0.06773	0.10790	1.59
4096	0.26188	0.51079	1.95
8192	1.05175	2.52066	2.40

Para $n = 256$ ambas versiones tardan prácticamente lo mismo (incluso la Versión 2 es ligeramente más rápida), ya que la matriz completa cabe sin problema en la caché y el orden de acceso todavía no penaliza el desempeño. A partir de $n = 512$ la Versión 1 (por filas) es consistentemente más rápida, y la brecha entre ambas crece de forma sostenida con n : la razón v_2/v_1 pasa de 1,25 en $n = 512$ a 2,40 en $n = 8192$. Esto confirma lo señalado en la Sección 1.3: como el número de operaciones ($O(n^2)$) es el mismo para las dos versiones, el crecimiento de la brecha de tiempos se explica por el aumento de los fallos de caché en la Versión 2 a medida que la matriz deja de caber en los niveles superiores de la jerarquía de memoria (L1/L2), mientras que la Versión 1 mantiene un patrón de acceso contiguo y sigue aprovechando la localidad espacial.

1.5. Resultados experimentales

La Figura 1 muestra gráficamente los tiempos de la Tabla 1 en función del tamaño n .

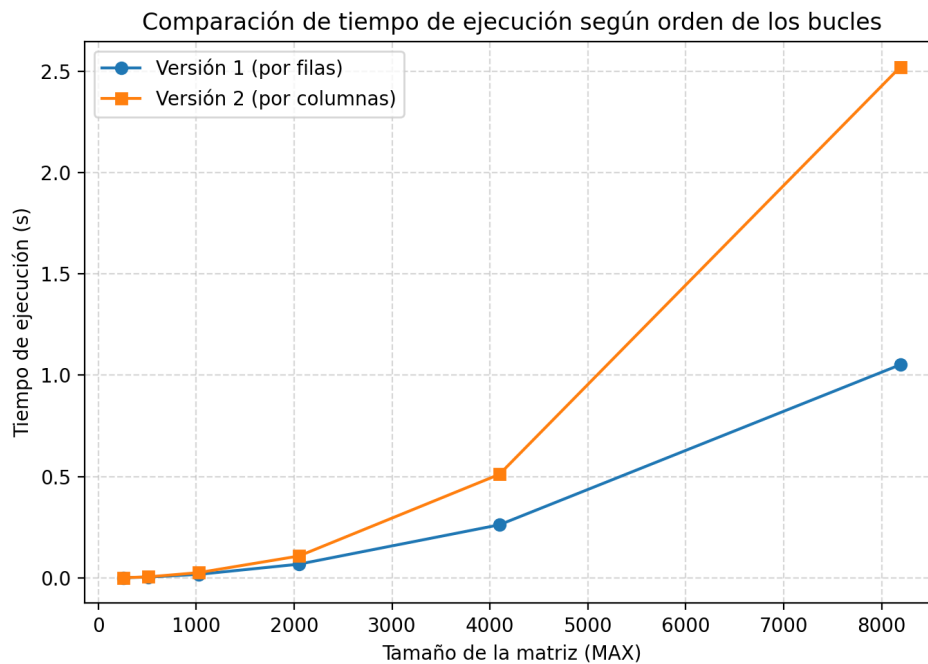


Figura 1: Tiempo de ejecución de ambas versiones en función de MAX.

Ambas curvas crecen de manera consistente con la complejidad cuadrática $O(n^2)$ esperada, pero la curva de la Versión 2 se separa cada vez más de la Versión 1 a medida que n aumenta, en lugar de mantenerse paralela. Este comportamiento superlineal en la brecha (y no solo en el tiempo absoluto) es la evidencia experimental de que el costo real no depende únicamente del número de operaciones aritméticas, sino también del *movimiento de datos* entre la memoria principal y la caché: al recorrer A por columnas, cada acceso a $A[i][j]$ en la Versión 2 tiende a traer a la caché una línea completa que en su mayoría no se reutiliza antes de ser desalojada, generando más fallos de caché por cada elemento útil accedido. La Versión 1, al recorrer A en el mismo orden en que se almacena en memoria (por filas, en C/C++), reutiliza cada línea de caché traída desde memoria principal, lo que explica su menor tiempo de ejecución para todos los tamaños evaluados, especialmente a partir de $n = 1024$.

2. Multiplicación clásica de matrices

2.1. Implementación

Se implementó el algoritmo clásico de multiplicación de matrices $C = A \times B$, con A, B, C de tamaño $n \times n$, utilizando tres bucles anidados en orden i - j - k .

Listing 3: Multiplicación clásica de matrices (tres bucles anidados)

```

1 // C = A * B, con A, B, C de tamaño n x n
2 void multiplicacionClasica(const vector<vector<double>>& A,
3                           const vector<vector<double>>& B,
4                           vector<vector<double>>& C) {
5     int n = A.size();
6     for (int i = 0; i < n; i++) {
7         for (int j = 0; j < n; j++) {
8             double suma = 0.0;
9             for (int k = 0; k < n; k++) {
10                 suma += A[i][k] * B[k][j];
11             }
12             C[i][j] = suma;
13         }
14     }
15 }

```

2.2. Ejecución con diferentes tamaños y medición de tiempo

Para evaluar el desempeño se ejecutó el algoritmo sobre matrices cuadradas generadas aleatoriamente, con n igual a 64, 128, 256, 512, 1024 y 1536, elegidos para cubrir órdenes de magnitud distintos y así observar con claridad el crecimiento del tiempo de ejecución. Para cada tamaño se promediaron 2 repeticiones y el tiempo se midió únicamente sobre el ciclo de multiplicación. Los resultados se resumen en la Tabla 2.

2.3. Relación entre el tamaño de la matriz y el tiempo de ejecución

La Figura 2 muestra un crecimiento del tiempo de ejecución mucho más pronunciado que el observado en la Sección 1 para los bucles simples: mientras que allí el tiempo crecía de forma cuadrática, aquí el tiempo pasa de 0,005 s en $n = 64$ a 87,15 s en $n = 1536$, es decir, un aumento de más de 17000 veces para un incremento de tamaño de solo 24 veces.

Para verificar si este crecimiento es consistente con un comportamiento cúbico ($O(n^3)$), se puede estimar el exponente de crecimiento comparando los extremos de la tabla:

Cuadro 2: Tiempo de ejecución de la multiplicación clásica según tamaño de n

n	Tiempo (s)	Factor respecto al tamaño anterior
64	0.00503	-
128	0.06115	$\times 12,16$ ($n \times 2$)
256	0.25892	$\times 4,23$ ($n \times 2$)
512	2.12656	$\times 8,21$ ($n \times 2$)
1024	21.2528	$\times 10,00$ ($n \times 2$)
1536	87.1471	$\times 4,10$ ($n \times 1,5$)

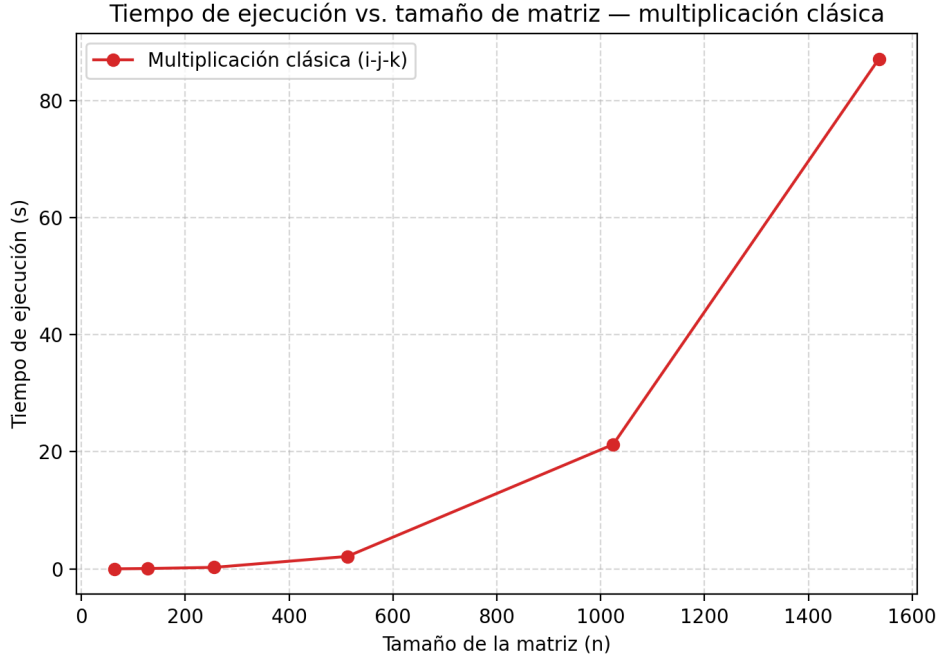


Figura 2: Tiempo de ejecución de la multiplicación clásica en función de n .

$$\frac{T(1536)}{T(64)} \approx 17332, \quad \left(\frac{1536}{64}\right)^3 = 24^3 = 13824 \quad (3)$$

Los valores son del mismo orden de magnitud, y el exponente empírico p que satisface $17332 = 24^p$ es $p \approx 3,07$, muy cercano al 3 teórico. La pequeña diferencia (un exponente ligeramente mayor a 3) es consistente con el efecto adicional de la caché: a medida que n crece, las matrices dejan de caber en los niveles superiores de la jerarquía de memoria, por lo que el tiempo real crece un poco más rápido que el número puro de operaciones aritméticas.

Al observar los factores de crecimiento por cada duplicación de n (columna derecha de la Tabla 2), se espera en teoría un factor cercano a $2^3 = 8$. Esto se cumple razonablemente bien entre $n = 512$ y $n = 1024$ (factor 10,00), mientras que entre $n = 64$ y $n = 128$ el factor medido (12,16) es menos confiable porque los tiempos absolutos son muy pequeños (milisegundos) y están más expuestos al ruido de medición y a la resolución del reloj del sistema. El crecimiento se vuelve claramente notorio y costoso a partir de $n = 512$, donde el tiempo ya supera los 2 segundos, y se dispara a partir de $n = 1024$, superando los 20 segundos.

2.4. Complejidad algorítmica

El algoritmo recorre tres bucles anidados, cada uno de longitud n : el bucle en i recorre las filas de A/C , el bucle en j recorre las columnas de B/C , y el bucle en k calcula el producto punto entre la fila i de A y la columna j de B . El número total de operaciones multiplicación-suma realizadas es:

$$N_{\text{op}} = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = n \times n \times n = n^3 \quad (4)$$

Por lo tanto, la complejidad temporal del algoritmo clásico de multiplicación de matrices es:

$$T(n) = O(n^3) \quad (5)$$

A diferencia del análisis de bucles de la Sección 1, aquí el crecimiento cúbico no es solo un efecto de caché: proviene directamente del número de operaciones aritméticas necesarias, ya que cada uno de los n^2 elementos de C requiere n multiplicaciones y $n-1$ sumas para calcularse. Esto implica que, en el caso clásico, el tiempo de ejecución crece de forma mucho más agresiva con n que en el ejemplo de la Sección 1 ($O(n^2)$), lo que también se espera confirmar experimentalmente en la Sección 2.3.

3. Multiplicación de matrices por bloques

3.1. Fundamento de la multiplicación por bloques

La multiplicación por bloques consiste en dividir las matrices A , B y C en submatrices cuadradas de tamaño $bs \times bs$ (bloques), y calcular el producto $C = A \times B$ procesando esos bloques uno a la vez en lugar de recorrer las matrices completas elemento por elemento. Al elegir bs de forma que un bloque de A , uno de B y uno de C quepan simultáneamente en la caché (idealmente en L1 o L2), todos los accesos necesarios para completar ese bloque se resuelven casi siempre desde la caché y no desde la memoria principal. Esto aumenta la localidad temporal: cada dato cargado se reutiliza varias veces mientras se procesa su bloque, antes de ser desalojado, a diferencia del algoritmo clásico, donde para matrices grandes cada elemento se puede traer y desechar de la caché muchas veces a lo largo de todo el cálculo.

3.2. Implementación

La implementación utiliza seis bucles anidados: los tres primeros (ii , jj , kk) recorren los bloques con paso bs , y los tres internos (i , j , k) recorren los elementos dentro de cada bloque, acumulando sobre $C[i][j]$ igual que en la versión clásica.

Listing 4: Multiplicación por bloques (seis bucles anidados)

```
1 // C = A * B, con A, B, C de tamaño n x n, dividido en bloques de tamaño bs
2 // Se asume que C ya está inicializada en 0 antes de llamar a esta función.
3 void multiplicacionPorBloques(const vector<vector<double>>& A,
4                               const vector<vector<double>>& B,
5                               vector<vector<double>>& C,
6                               int bs) {
7     int n = A.size();
8
9     for (int ii = 0; ii < n; ii += bs) {
10         for (int jj = 0; jj < n; jj += bs) {
11             for (int kk = 0; kk < n; kk += bs) {
12
13                 int iMax = min(ii + bs, n);
```

```

14     int jMax = min(jj + bs, n);
15     int kMax = min(kk + bs, n);
16
17     for (int i = ii; i < iMax; i++) {
18         for (int j = jj; j < jMax; j++) {
19             double suma = C[i][j];
20             for (int k = kk; k < kMax; k++) {
21                 suma += A[i][k] * B[k][j];
22             }
23             C[i][j] = suma;
24         }
25     }
26 }
27 }
28 }
29 }
30 }

```

3.3. Evaluación con diferentes tamaños de matriz

Cuadro 3: Tiempo de ejecución: clásica vs. por bloques (bloque fijo $bs = 64$) según n

n	Clásica (s)	Bloques (s)	Mejora
64	0.00516	0.00622	-20,7 %
128	0.04249	0.03781	+11,0 %
256	0.27310	0.27478	-0,6 %
512	2.17642	2.08098	+4,4 %
1024	20.128	16.3326	+18,9 %
1536	89.6297	54.4694	+39,2 %

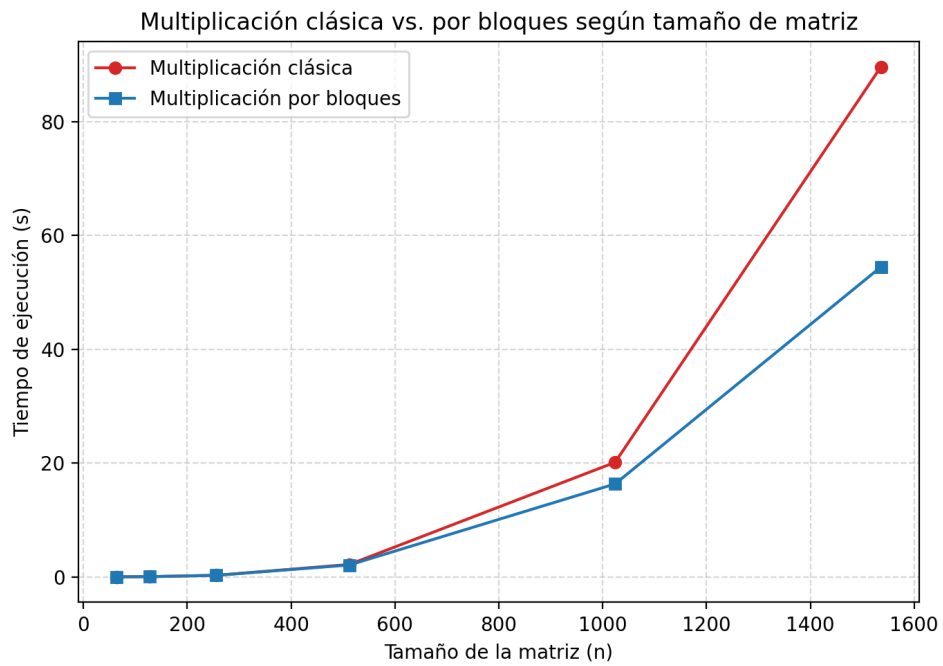


Figura 3: Tiempo de ejecución de la multiplicación clásica y por bloques en función de n .

Para $n \leq 256$ la versión por bloques no ofrece ninguna ventaja clara e incluso puede ser ligeramente más lenta: con matrices tan pequeñas, todo el problema ya cabe cómodamente en la caché aunque se use la versión clásica, por lo que dividir en bloques solo agrega el costo extra de los tres bucles externos sin aportar ningún beneficio de localidad adicional. A partir de $n = 512$ la ventaja de la versión por bloques empieza a notarse y crece de forma sostenida: $+4,4\%$ en $n = 512$, $+18,9\%$ en $n = 1024$ y $+39,2\%$ en $n = 1536$. Esto confirma que, tal como se explica en la Sección 3.6, ambas versiones mantienen la misma complejidad $O(n^3)$ (ambas curvas siguen creciendo de forma muy similar en la Figura 3), pero la brecha entre ellas se ensancha con n porque el beneficio de reutilizar datos en caché se vuelve más relevante justamente cuando las matrices dejan de caber en la jerarquía de memoria superior, que es donde la versión clásica empieza a sufrir más fallos de caché.

3.4. Experimentación con diferentes tamaños de bloque

Cuadro 4: Tiempo de ejecución según tamaño de bloque ($n = 1024$ fijo)

Tamaño de bloque	Tiempo (s)
8	18.8084
16	16.7028
32	16.8704
64	16.3579
128	16.0277
256	16.0511
512	17.3551
1024	20.0037

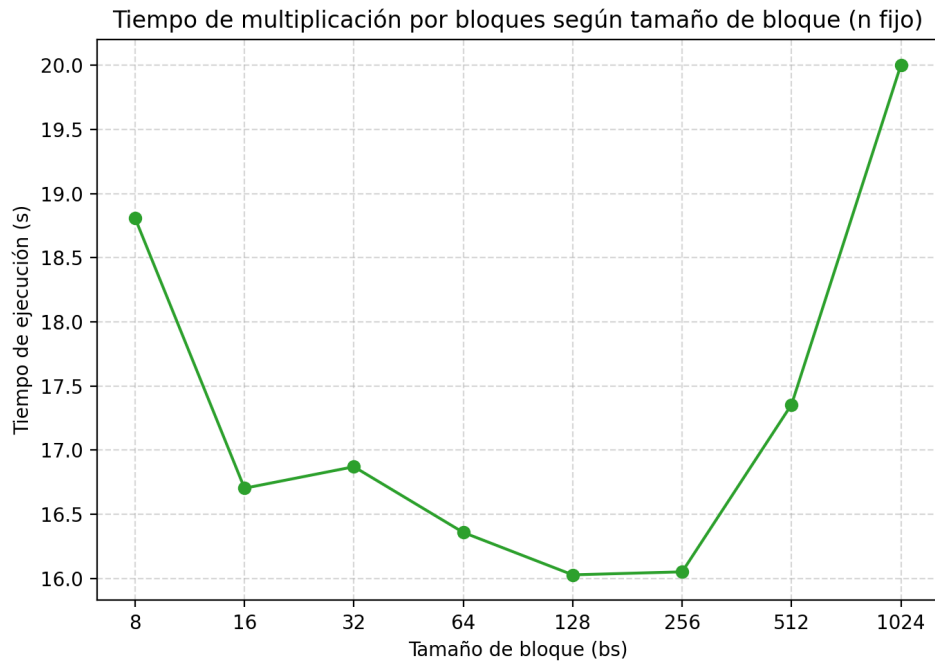


Figura 4: Tiempo de ejecución de la multiplicación por bloques según el tamaño de bloque, para $n = 1024$ fijo.

La curva de la Figura 4 tiene forma de “U”: el tiempo es alto en los extremos y mínimo en

una zona intermedia. Con bloques muy pequeños ($bs = 8, 18, 81$ s) el bloque cabe de sobra en la caché, pero el número de iteraciones de los tres bucles externos crece mucho, y el overhead de controlar tantos bucles pequeños empieza a dominar sobre el beneficio de la localidad. En el otro extremo, $bs = 1024$ (es decir, $bs = n$, sin división real en bloques) da 20,00 s, un valor casi idéntico al de la multiplicación clásica en $n = 1024$ (20,128 s de la Tabla 3), lo cual confirma que con $bs = n$ el algoritmo por bloques degenera exactamente en el algoritmo clásico.

El mejor desempeño se obtiene en $bs = 128$ (16,0277 s), prácticamente empatado con $bs = 256$ (16,0511 s, diferencia menor al 0,2 %), lo que sugiere que el tamaño de bloque óptimo para esta máquina está en ese rango. Esto es consistente con el fundamento explicado en la Sección 3.1: valores de bs en ese rango son lo suficientemente grandes para amortizar el overhead de los bucles externos, pero todavía lo suficientemente pequeños para que los bloques de A , B y C involucrados quepan en la caché (L1 y/o L2) y se reutilicen varias veces antes de ser desalojados.

3.5. Comparación con la multiplicación clásica

La Tabla 3 y la Figura 3 muestran que, con un bloque fijo de $bs = 64$, la multiplicación por bloques iguala o supera a la clásica a partir de $n = 512$, y la mejora crece con el tamaño de la matriz: de prácticamente nula en $n = 512$ (+4,4 %) a una reducción de tiempo de casi el 40 % en $n = 1536$ (89,63 s frente a 54,47 s).

Si en lugar de $bs = 64$ se usa el bloque óptimo encontrado en la Sección 3.4 ($bs = 128$) para $n = 1024$, la mejora respecto a la clásica sube de +18,9 % (con $bs = 64$) a $\frac{20,128 - 16,0277}{20,128} \approx +20,4$ %. La diferencia entre usar $bs = 64$ y el $bs = 128$ óptimo es pequeña en este caso, pero confirma que la elección del tamaño de bloque sí influye en cuánta ventaja obtiene realmente la versión por bloques sobre la clásica: un bloque mal elegido (muy chico o muy grande, como se vio en la Sección 3.4) puede reducir esa ventaja o incluso eliminarla por completo.

3.6. ¿Por qué la versión por bloques puede tener mejor desempeño con la misma complejidad algorítmica?

Tanto la multiplicación clásica como la multiplicación por bloques calculan exactamente el mismo conjunto de productos y sumas: dividir en bloques no elimina ni agrega operaciones aritméticas, solo cambia el orden en que se visitan. Por eso ambas tienen la misma complejidad computacional $O(n^3)$ (Sección 2.4).

La diferencia de desempeño se explica por la complejidad del movimiento de datos, no por el número de operaciones:

- En la versión clásica, cuando n es grande, las filas y columnas de A y B que participan en el cálculo dejan de caber en la caché. Esto provoca que muchos valores se traigan repetidamente desde memoria principal a lo largo del recorrido de los tres bucles, generando un alto número de fallos de caché.
- En la versión por bloques, al restringir el cálculo a submatrices de tamaño $bs \times bs$ que sí caben en la caché, cada bloque de A y de B se carga una vez y se reutiliza para calcular todo el bloque correspondiente de C antes de pasar al siguiente. Esto reduce drásticamente el número de accesos a memoria principal por cada operación aritmética útil.

En otras palabras, ambos algoritmos son equivalentes en *cuántas* operaciones hacen ($O(n^3)$), pero muy distintos en *cuántos datos mueven* entre la memoria principal y la caché para hacerlas. Como el acceso a memoria principal es varios órdenes de magnitud más lento que un acceso a caché, ese movimiento de datos suele ser el verdadero cuello de botella en problemas de este tamaño, y es lo que explica que la versión por bloques sea más rápida en la práctica a pesar de tener la misma complejidad algorítmica asintótica.

4. Análisis del movimiento de datos y memoria caché

4.1. Acceso a los elementos de las matrices

En la multiplicación clásica (Listado 3), para cada par (i, j) el bucle interno en k accede a:

- $A[i][k]$: con k variando y i fijo, se recorre la fila i de A en orden, es decir, direcciones de memoria consecutivas (ya que en C/C++ las matrices se almacenan por filas).
- $B[k][j]$: con k variando y j fijo, se recorre la columna j de B , es decir, elementos separados por n posiciones en memoria (un “salto” de tamaño n en cada iteración).
- $C[i][j]$: la misma dirección de memoria se reutiliza en las n iteraciones del bucle en k (se va acumulando en la variable `suma` antes de escribirse una sola vez en $C[i][j]$).

En la multiplicación por bloques (Listado 4) el patrón es el mismo a nivel de instrucción ($A[i][k]$, $B[k][j]$, $C[i][j]$), pero los índices i , j , k ya no recorren toda la matriz de una sola pasada: quedan acotados al rango $[ii, ii + bs)$, $[jj, jj + bs)$, $[kk, kk + bs)$ de cada bloque antes de avanzar al siguiente. Es decir, se hacen exactamente los mismos accesos elemento por elemento, pero agrupados y reordenados por bloques.

4.2. Diferencias en el patrón de acceso a memoria

La diferencia clave no está en *qué* elementos se acceden (son los mismos n^3 pares de accesos en ambos casos), sino en el tamaño del conjunto de trabajo (*working set*) que está “activo” en cada instante:

- En la versión clásica, mientras se recorre una fila completa de C (bucle en j), el algoritmo puede necesitar acceder a prácticamente toda la matriz B (n^2 elementos) antes de volver a reutilizar cualquiera de sus datos. Para n grande, B completa no cabe en la caché, por lo que muchos de esos accesos son fallos de caché.
- En la versión por bloques, en un instante dado el algoritmo solo trabaja con tres bloques de tamaño $bs \times bs$ (uno de A , uno de B y uno de C). Si bs se elige de forma que esos tres bloques quepan en la caché, todos los accesos dentro de ese bloque (los tres bucles internos) se resuelven casi siempre desde la caché, y el bloque completo se reutiliza antes de pasar al siguiente.

4.3. Localidad espacial y temporal

- Localidad espacial: tendencia de un programa a acceder a direcciones de memoria cercanas entre sí en poco tiempo. Se manifiesta en el acceso a $A[i][k]$, que recorre una fila completa de forma contigua, aprovechando que cada línea de caché traída desde memoria principal contiene varios elementos consecutivos que sí se van a usar a continuación.
- Localidad temporal: tendencia de un programa a volver a acceder a la misma dirección de memoria varias veces en poco tiempo. Se manifiesta en $C[i][j]$ (se reutiliza en las n iteraciones del bucle en k) y, de forma mucho más marcada, en la versión por bloques, donde cada bloque de A y de B se reutiliza en múltiples productos internos antes de ser desalojado de la caché.

La versión clásica solo explota bien la localidad espacial en el acceso a A , y la localidad temporal se limita a la variable acumuladora `suma/C[i][j]`. La versión por bloques, en cambio, agrega una fuerte localidad temporal adicional a nivel de bloque: cada elemento de A y B dentro de un bloque se reutiliza aproximadamente bs veces (una por cada combinación de i , j dentro del bloque) mientras permanece en caché, en lugar de una sola vez.

4.4. Por qué la versión por bloques aprovecha mejor la caché

Como se explicó en la Sección 3.6, ambas versiones realizan el mismo número de operaciones aritméticas. La ventaja de la versión por bloques viene de que, al limitar el trabajo a submatrices de tamaño $bs \times bs$, el conjunto de datos “activo” en un momento dado se puede mantener dentro de la caché durante todo el procesamiento de ese bloque. Esto significa que cada dato que se trae desde la memoria principal se usa muchas veces (del orden de bs veces) antes de ser desalojado, en lugar de una sola vez como tiende a ocurrir en la versión clásica cuando n excede la capacidad de la caché. El resultado es una reducción directa en el número de transferencias entre memoria principal y caché, sin reducir el número de operaciones aritméticas.

4.5. Complejidad computacional vs. costo de movimiento de datos

La complejidad computacional (número de operaciones aritméticas) de ambas versiones es $O(n^3)$, como se determinó en la Sección 2.4, y no depende de cómo se organicen los bucles. Sin embargo, el costo de movimiento de datos (cuántas veces hay que transferir datos entre la memoria principal y la caché) sí depende del orden de acceso:

- En la versión clásica, en el peor caso (cuando B no cabe en caché) cada uno de los n^3 accesos a $B[k][j]$ puede traducirse en una transferencia desde memoria principal, por lo que el costo de movimiento de datos puede acercarse también a $O(n^3)$.
- En la versión por bloques, cada bloque de tamaño $bs \times bs$ se transfiere una vez a la caché y se reutiliza en, aproximadamente, bs operaciones. Esto reduce el número de transferencias a un orden aproximado de $O(n^3/bs)$: mientras más grande el bloque (sin dejar de caber en caché), menos transferencias por operación útil.

Esta es la razón formal por la que dos algoritmos con la misma complejidad computacional $O(n^3)$ pueden tener tiempos de ejecución muy distintos en la práctica: el tiempo real no depende solo de cuántas operaciones aritméticas se hacen, sino también de cuántas de ellas requieren esperar un acceso a memoria principal (mucho más lento que un acceso a caché).

4.6. Relación con los resultados experimentales

Los resultados de la Sección 3 son consistentes con este análisis.

En la Figura 5 (y la Tabla 3), para $n \leq 256$ la versión por bloques no ofrece ninguna ventaja porque, en esos tamaños, incluso la versión clásica trabaja con matrices que ya caben en la caché, por lo que el movimiento de datos no es el cuello de botella y no hay margen para mejorar. A partir de $n = 512$, cuando la matriz completa empieza a exceder la capacidad de la caché, la ventaja de la versión por bloques crece de forma sostenida (+4,4 %, +18,9 % y +39,2 % en $n = 512, 1024$ y 1536 respectivamente), justo como predice el análisis: a mayor n , mayor es el número de fallos de caché evitados por la reutilización de bloques.

De forma similar, la forma en “U” observada en la Figura 4 (Sección 3.4) también encaja con la jerarquía de memoria: bloques demasiado chicos ($bs = 8$) no amortizan bien el overhead de control de los bucles aunque casi no generen fallos de caché, mientras que bloques demasiado grandes ($bs = 512$ o $bs = 1024$) dejan de caber en los niveles más rápidos de la caché y vuelven a comportarse como la versión clásica, perdiendo la ventaja de la reutilización. El punto óptimo (entre $bs = 128$ y $bs = 256$) es precisamente el rango donde el bloque es grande para amortizar overhead pero pequeño para seguir residiendo en caché, confirmando experimentalmente el comportamiento esperado de la jerarquía de memoria.

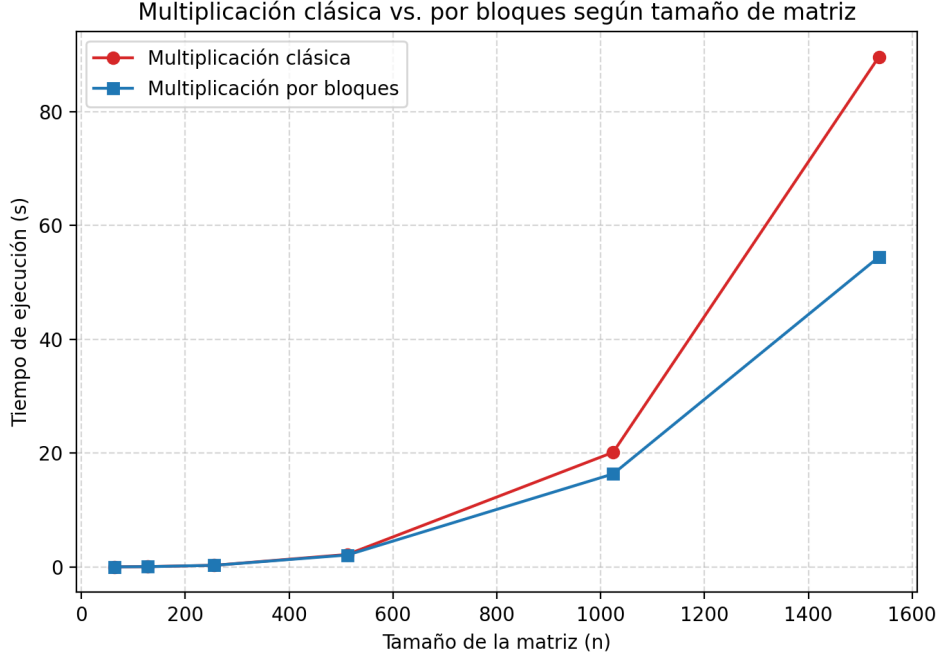


Figura 5: Tiempo de ejecución de la multiplicación clásica y por bloques en función de n .

5. Evaluación utilizando Valgrind y KCachegrind

5.1. Metodología

Se utilizó Valgrind con la herramienta Cachegrind para simular el comportamiento de la memoria caché (L1 de instrucciones, L1 de datos y la caché de último nivel, LL) durante la ejecución de la multiplicación clásica y de la multiplicación por bloques, y KCachegrind para visualizar e inspeccionar los resultados generados.

Para que la comparación fuera directa, se usó el mismo tamaño de matriz ($n = 512$) y la misma semilla aleatoria en ambas versiones, ejecutando una sola multiplicación (sin repeticiones, a diferencia de los experimentos de tiempo de las Secciones 2 y 3) para no inflar innecesariamente el análisis. La versión por bloques se ejecutó con $bs = 128$, el tamaño de bloque óptimo encontrado en la Sección 3.4. Los comandos utilizados fueron:

Listing 5: Comandos utilizados para el análisis con Cachegrind

```
1 valgrind --tool=cachegrind --cache-sim=yes ./mult_clasica_valgrind
2 valgrind --tool=cachegrind --cache-sim=yes ./mult_bloques_valgrind
```

5.2. Resultados comparativos

La Tabla 5 resume las métricas obtenidas por Cachegrind para ambas versiones.

Como se observa en la Figura 6, la diferencia entre ambas versiones se concentra casi por completo en la caché L1 de datos (paneles izquierdo y central), mientras que los fallos de último nivel (panel derecho) son prácticamente idénticos.

5.3. Análisis crítico

Instrucciones ejecutadas. La versión por bloques ejecuta ligeramente *más* instrucciones que la clásica (+1,1 %), lo cual es esperable: los tres bucles externos que recorren los bloques (ii , jj ,

Cuadro 5: Comparación de instrucciones, accesos a memoria y fallos de caché (Cachegrind, $n = 512$)

Métrica	Clásica	Bloques ($bs = 128$)	Variación
Instrucciones ejecutadas (I refs)	1,132,032,699	1,144,729,186	+1,1 %
Accesos a datos (D refs)	414,666,680	420,753,669	+1,5 %
Fallos L1 instrucciones (I1 misses)	2,358	2,364	≈ 0 %
Fallos LL instrucciones (LLi misses)	2,223	2,227	≈ 0 %
Fallos L1 datos (D1 misses)	70,615,163	17,766,341	-74,8 %
Tasa de fallos L1 datos	17,0 %	4,2 %	-
Fallos LL datos (LLd misses)	108,312	108,313	≈ 0 %

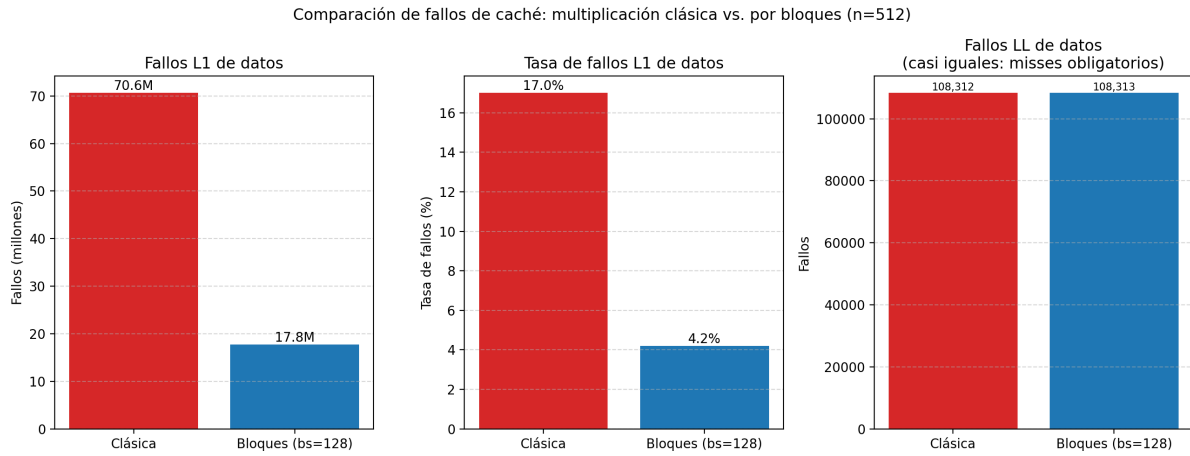


Figura 6: Fallos de caché L1 de datos (cantidad y tasa) y fallos de último nivel (LL) para la multiplicación clásica y por bloques.

kk) y el cálculo de los límites ($iMax$, $jMax$, $kMax$) agregan trabajo de control que la versión clásica no tiene. Esto confirma, a nivel de instrucciones reales ejecutadas, lo mismo que se argumentó teóricamente en la Sección 3.6: dividir en bloques no reduce el número de operaciones, e incluso agrega una pequeña sobrecarga.

Accesos a memoria de datos. De forma similar, los accesos totales a datos (D refs) también son ligeramente mayores en la versión por bloques (+1,5 %), por la misma razón: más instrucciones de control implican también algunos accesos adicionales a variables de control.

Fallos de caché de instrucciones. Tanto I1 misses como LLi misses son prácticamente idénticos entre ambas versiones (diferencias de apenas 4-6 fallos sobre miles). Esto tiene sentido: el tamaño del código de ambas funciones es pequeño y similar, por lo que no hay diferencias relevantes en el uso de la caché de instrucciones.

Fallos de caché de datos: el resultado clave. Aquí es donde aparece la diferencia más importante de todo el laboratorio: los fallos en la caché L1 de datos caen de 70,615,163 en la versión clásica a 17,766,341 en la versión por bloques, una reducción del 74,8 %, y la tasa de fallos L1 baja de 17,0 % a 4,2 %. Esto es exactamente lo que predice el análisis de localidad de la Sección 4: al trabajar con bloques de 128×128 elementos que caben en la caché, cada dato cargado se reutiliza muchas veces antes de ser desalojado, mientras que la versión clásica recorre columnas completas de B (elementos separados por $n = 512$ posiciones) y termina desalojando y volviendo a cargar los mismos datos repetidamente.

Fallos de caché de último nivel (LL). Llama la atención que LLd **misses** sea prácticamente idéntico en ambas versiones (108,312 vs. 108,313). Esto no contradice el análisis: los fallos en la caché de último nivel representan, en gran medida, los fallos *obligatorios* (*compulsory misses*): la primera vez que cada dato de las matrices A y B debe traerse desde memoria principal, algo que ambas versiones necesitan hacer exactamente una vez por elemento, sin importar el orden de acceso. La diferencia entre ambas versiones no está en *cuántos* datos distintos se traen desde memoria principal (eso es igual, porque procesan el mismo conjunto de datos), sino en *cuántas veces* esos datos, una vez en la caché, deben volver a traerse porque fueron desalojados antes de reutilizarse. Esa es precisamente la diferencia que capturan los fallos de L1, no los de LL.

Relación con la complejidad y con los tiempos medidos. Este resultado conecta directamente los tres puntos centrales del laboratorio:

- Ambas versiones tienen la misma complejidad computacional $O(n^3)$ (Sección 2.4) y, de hecho, la versión por bloques ejecuta ligeramente *más* instrucciones, no menos.
- Sin embargo, la versión por bloques genera un 74,8% menos fallos de caché L1 de datos, es decir, mueve muchos menos datos entre la caché L1 y los niveles superiores de la jerarquía de memoria para realizar el mismo trabajo.
- Esto es consistente con las mediciones de tiempo de la Sección 3.3, donde para $n = 512$ la versión por bloques ya era más rápida que la clásica, y esa ventaja seguía creciendo con n . El experimento con Cachegrind muestra la causa exacta detrás de esa ventaja: no es que se hagan menos operaciones (se hacen casi las mismas, incluso un poco más), sino que se accede a la memoria de forma mucho más eficiente.

En conjunto, estos resultados confirman experimentalmente la conclusión central del laboratorio: cuando la complejidad computacional de dos algoritmos es la misma, el tiempo de ejecución real puede seguir siendo muy distinto, y esa diferencia se explica por el patrón de acceso a memoria y su interacción con la jerarquía de caché, no por el número de operaciones aritméticas realizadas.

Repositorio

El código fuente completo utilizado en este laboratorio se encuentra disponible en el siguiente repositorio:

https://github.com/MarceloMAQD/lab2_computacion_paralela